

SentinelTrie

Conclusiones y propuestas de trabajo futuro

Disseny i Anàlisi d'Algoritmes Avancats - ENTI-UB | Antonio Malaga - Max Mora - Joan Cobos

1. Resumen ejecutivo

SentinelTrie es un motor de detección de malware por firmas implementado en Python que utiliza un **Arbol Trie** como estructura central. El sistema cumple los cinco principios técnicos obligatorios del enunciado: trata un problema real de ciberseguridad (detección de código malicioso por firma), utiliza una estructura de datos no trivial y justificada (Trie), aplica recursividad en múltiples puntos (inserción, búsqueda exacta y aproximada, recorrido de directorios), incorpora programación orientada a objetos con polimorfismo real (MotorEscaneo -> EscaneoFirmaExacta/Parcial) y se ha desarrollado siguiendo gitflow con ramas por funcionalidad.

2. Que hemos aprendido

2.1. Sobre la estructura de datos

Antes del proyecto teníamos una idea abstracta de que un Trie ahorra memoria gracias a los prefijos compartidos, pero verlo empíricamente sobre nuestra propia base de datos lo confirma: las cuatro variantes de la familia *Trojan.Family.X.v1..v4* ocupan 11 nodos compartidos + 4 ramas, frente a $4 \times 11 = 44$ bytes duplicados en un almacenamiento plano. En la base de datos de demo el ratio nodos/firmas es 8.33 (frente a una longitud media de firma de ~10 bytes), lo que se traduce en un ahorro de memoria del ~17 %.

2.2. Sobre el polimorfismo

El diseño con una clase abstracta *MotorEscaneo* simplificó enormemente el código cliente. La clase *Escaner* y la CLI principal nunca preguntan qué tipo de motor reciben - se limitan a invocar *escanear_archivo()*. Esto facilitó añadir el modo parcial sin tocar una sola línea del resto del sistema; añadir un futuro modo basado en YARA o Aho-Corasick sería igualmente trivial.

2.3. Sobre la recursividad

Las operaciones del Trie son el ejemplo canónico de problema donde la recursividad es claramente la solución: el código queda más corto que la versión iterativa equivalente y es más fácil de razonar. El backtracking del modo parcial sería muy engorroso de escribir sin recursión. Por contra, observamos que para profundidades grandes (>1000 firmas con prefijos largos) Python lanza *RecursionError*; mitigar esto requeriría *sys.setrecursionlimit* o reescribir como iteración con pila explícita.

2.4. Sobre los límites del enfoque por firmas

Implementando el motor parcial constatamos en primera persona la razón de fondo por la que los antivirus comerciales están abandonando la detección exclusivamente por firmas: basta cambiar 1 byte en una muestra para que el motor exacto deje de encontrarla. El motor parcial mitiga esto a costa de explotar la complejidad. Es un dilema clásico del campo: o se renuncia a atrapar

mutaciones, o se renuncia a la velocidad. Las soluciones modernas combinan firmas (rapidas pero rigidas) con analisis heuristico y machine learning.

3. Limitaciones de la implementacion actual

Documentamos las limitaciones conocidas con honestidad academica:

- **Solo deteccion estatica.** El sistema no analiza el comportamiento en ejecucion del binario, ni desempaqueta archivos comprimidos. Un malware empaquetado con UPX o cifrado no sera detectado aunque su firma este en la BD.
- **Firmas literales sin comodines.** Las firmas reales del sector (ClamAV, YARA) admiten patrones como `4D 5A ?? ?? E8`. Soportar comodines requiere extender el Trie con un campo `byte_comodin` y un fork en `buscar_en_datos`.
- **Limite de 10 MB por archivo.** Para evitar consumo desbocado de RAM, `max_bytes` esta capeado a 10 MB. Archivos mayores se leen truncados. Una mejora consistiria en escaneo por *chunks* manteniendo el estado entre lecturas.
- **Sin cuarentena.** Cuando se detecta una amenaza solo se reporta - no se mueve el archivo a una zona segura ni se elimina. El alcance del proyecto se centra en el motor de deteccion.
- **Dependencia del orden de hijos en el modo parcial.** El backtracking visita los hijos en orden de insercion. Una poda mas agresiva (ordenando por probabilidad de matching) reduciria el espacio explorado.

4. Propuestas de trabajo futuro

Las propuestas se han ordenado por impacto esperado / esfuerzo. Las marcadas como **CORTO PLAZO** son implementables en unas horas; las de **MEDIO PLAZO** requieren rediseño; las de **LARGO PLAZO** exceden el alcance academico.

4.1. Corto plazo

- Soporte de comodines simples (??) en las firmas.
- Logger estructurado (JSON) con niveles configurables.
- Hashing previo (MD5/SHA1) de archivos pequenos para cortocircuitar el escaneo si el hash ya esta en una blacklist.
- Modo `--watch` con *watchdog* para vigilar directorios en tiempo real.
- Exportacion de informe en formato HTML autocontenido.

4.2. Medio plazo

- Migracion del nucleo al algoritmo **Aho-Corasick** para obtener $O(m + z)$ en peor caso.
- Soporte completo de reglas YARA (parser de reglas + integracion con el motor existente como tercera subclase de *MotorEscaneo*).
- Persistencia binaria del Trie (formato propio o *marisa-trie*) para arranque inmediato con bases de datos grandes.
- Paralelizacion del recorrido de directorios con *ProcessPoolExecutor*.
- Suite de fuzzing con *hypothesis* para detectar casos extranos no cubiertos por unittest.

4.3. Largo plazo

- Reescritura del bucle de matching en Rust / C, exponiendo una API Python via PyO3 o ctypes (esperamos x15-x20).
- Modulo heuristico complementario que aprenda perfiles de comportamiento normal por archivo (ML supervisado o estadistica Bayesiana).
- Conexion automatica a feeds publicos de firmas (ClamAV CVD, MalwareBazaar) con actualizacion incremental del Trie.
- Daemon/servicio del sistema (systemd, launchd) que escuche eventos de creacion de archivos y los escanee al vuelo, similar a un EDR.
- Interfaz grafica minimalista (Textual o PyQt) para usuarios no tecnicos.

5. Reflexion final

El proyecto cumple su objetivo academico: demostrar que una estructura de datos clasica - el Arbol Trie - aplicada con criterio puede transformar un problema aparentemente intratable (buscar miles de patrones simultaneamente en un archivo) en una operacion practicamente lineal en el tamano del archivo. La experiencia ha sido especialmente util para interiorizar tres principios que volveran a aparecer en cualquier sistema de deteccion: **preprocesar los patrones** para amortizar el coste de las consultas, **aprovechar prefijos compartidos** para ahorrar memoria y **polimorfismo** para extender funcionalidad sin tocar el codigo cliente.

Mas alla del Trie concreto, el ejercicio ha consolidado nuestra intuicion sobre cuando elegir una estructura por sus garantias asintoticas. SentinelTrie no compite con ClamAV - pero entender como funciona por dentro lo que usan los antivirus reales es, creemos, el verdadero entregable del proyecto.